

SAGE - CODEX HANDOFF

Post-feasibility design expansion - revised with unique-participant chain semantics and optional visible mini-SAGE marker

Discussion boundary: This handoff covers design discussion after Ethan's checkpoint: *"Our last full commit was about 3 minutes ago. A nice little proof of feasibility"*. CODEX should review critically, reconcile with the current repository, update appropriate artifacts if warranted, and commit coherent changes.

Repository rule: The latest Git repository state remains authoritative. This is a design handoff, not a replacement source of truth.

1. Reference implementation strategy

Build two practical image editors as independent SAGE consumers: a PHP/JavaScript web editor and a Flutter/Dart editor. Keep SAGE libraries separable from application-specific UI/business code.

Use shared conformance vectors so independently written implementations must serialize, encode, decode, compare, and resolve the same records identically.

```
PHP encode -> JavaScript decode -> PASS
JavaScript encode -> Dart decode -> PASS
Dart encode -> Python decode -> PASS
Python encode -> PHP decode -> PASS
```

Later example/reference ports may include Python, C/C++, Java/Kotlin, and C#. The initial goal is interoperability evidence, not immediate feature parity.

2. PARTICIPANT_ID replaces AI-specific identity

PARTICIPANT_ID is preferred over **AI_ID**. A participant can be an AI model, software product/version, editor, camera/capture system, transcoder, platform, or other participating implementation.

A participant does not have to equal a company. One vendor may register multiple participant IDs for different models, products, versions, or systems. SAGE need not model that vendor's internal hierarchy.

3. Participant payload simplification

The discussion moved from **PARTICIPANT_ID** + **TOOL_ID** + **EVENT_ID** + extension data toward a deliberately generic participant-owned payload:

```
PARTICIPANT_ENTRY {
  PARTICIPANT_ID
  EXT_DATA_1
  EXT_DATA_2
  EXT_DATA_3
}
```

Only **PARTICIPANT_ID** has globally standardized SAGE meaning. **EXT_DATA_1..3**, if retained, are optional bounded opaque participant-scoped values. Core may define binary representation, bounds, presence rules, preservation, and equality behavior, but not vendor/business meaning.

Examples include user ID, timestamp, copyright-holder claim, generation ID, capture ID, device token, model identifier, document ID, license reference, workflow reference, or nothing at all.

Trust boundary: extension values are participant-provided claims/data. SAGE does not authenticate their semantic truth merely by transporting them.

4. New chain rule: one entry per participant

This is the latest material chain change and should be evaluated explicitly. A PARTICIPANT_ID may appear at most once in the active SAGE participant chain.

SAGE is therefore not an event-by-event ledger. It records participating systems and their current relative last-touch order.

Canonical update behavior proposed:

```
IF current PARTICIPANT_ID already exists in CHAIN:  
  remove its existing entry  
  
construct/update current participant entry  
append current participant entry to end of CHAIN
```

The chain is ordered from **least-recent participant touch to most-recent participant touch** among currently represented unique participants. The final entry is the participant most recently represented by a conforming SAGE update.

Example:

```
Initial interactions: A -> B -> C  
A touches asset again: B -> C -> A  
C touches it again: B -> A -> C
```

This intentionally discards repeated event history for the same participant. A participant needing detailed history may maintain it internally or encode participant-defined information in EXT_DATA.

5. Timestamp decision

A mandatory SAGE timestamp is **not currently recommended**. The unique-participant ordered chain identifies relative last-touch order without requiring clock data.

Reasons include incorrect clocks, falsifiable timestamps, precision/timezone normalization issues, payload cost, and the risk consumers interpret participant-supplied time as SAGE-authenticated fact.

Participants remain free to use an EXT_DATA field for a timestamp.

"Each PARTICIPANT_ID MUST appear no more than once in the active chain. A conforming participant already represented MUST remove/replace its prior entry and append its updated entry at the end. Chain position represents relative most-recent SAGE-recorded participation, not authenticated wall-clock chronology."

6. Evidence semantics: deliberately modest guarantees

The current model should not claim complete provenance or authenticated custody. The strongest defensible interpretation is that SAGE may provide surviving evidence of which registered participants interacted with an asset and may identify the final participant represented in the surviving chain.

```
SAGE CAN REPORT:  
- recoverable SAGE evidence is PRESENT / DAMAGED / CONFLICT / NOT_DETECTED  
- which participant IDs survive in the recovered record  
- which participant is the final recorded entry  
  
SAGE CANNOT GUARANTEE:  
- every system that ever touched the asset is represented  
- the surviving chain is complete  
- the final recorded participant was literally the last software/device to touch the asset  
- participant extension data are truthful  
- a record was not replayed, copied, rewritten, or laundered
```

Recommended UI wording is therefore **"Final recorded participant"** or **"Latest SAGE participant"**, not "Last edited by."

Primary mission: determine whether lightweight recoverable evidence exists that a participating system generated, edited, or otherwise processed the asset. Full image-forensic reconstruction remains outside SAGE's scope.

7. SAGE registry: handshake and nothing more

The registry remains intentionally minimal and open-source. Its job is participant discovery, not provenance custody, certification, or vendor auditing.

```
PARTICIPANT_ID  
DISPLAY_NAME / VENDOR_NAME  
PUBLIC_URL  
STATUS
```

The participant-selected URL may describe the participant or lead to optional verification/documentation the participant operates. SAGE need not host generation records, prompts, camera serials, user relationships, copyright records, or asset histories.

"The SAGE Registry provides participant discovery only. Registration does not constitute certification, endorsement, authentication, or verification of participant claims or media provenance."

Private registration/account information such as email and authentication credentials is registry administration data and must not become embedded provenance.

8. Adoption pitch for conventional tools

Generic extension slots give Photoshop/GIMP/Paint-class tools, camera makers, CMS editors, and other non-AI software a reason to participate. They can use participant-owned fields for origin information without SAGE dictating a universal metadata vocabulary.

```
PARTICIPANT_ID = MY_IMAGE_TOOL  
EXT_DATA_1 = user ID  
EXT_DATA_2 = timestamp  
EXT_DATA_3 = copyright-holder claim
```

Another vendor can define all three differently. The adoption message is effectively: SAGE identifies your participant; the optional boxes are yours.

9. Optional visible mini-SAGE marker

New encoder idea: provide an optional visible marker that places a small SAGE QR-ish code in a corner of the exported image. This is intended as a human-visible / easily scannable supplement to the metadata and concealed layers.

Suggested encoder option:

```
visible_marker = true | false  
visible_marker_position = TOP_LEFT | TOP_RIGHT | BOTTOM_LEFT | BOTTOM_RIGHT
```

The visible marker should remain **optional** and should not alter the underlying logical SAGE participant record merely because it is displayed.

Because SAGE already uses a QR-ish concealed transport concept, the visible marker should use the same visual family rather than inventing a separate badge language.

A compact visible marker could encode only a tiny subset, such as SAGE magic/version plus the final recorded PARTICIPANT_ID, or another minimal profile-defined visible payload. CODEX should evaluate payload size and scan reliability before fixing the exact visible syntax.

The visible code is **not authoritative evidence by itself**. It is easy to crop, paint over, copy, or leave behind after hidden evidence is destroyed. A decoder must never treat the visible code as proof that concealed/metadata evidence still exists.

10. Three-channel evidence model

If the visible marker is adopted, SAGE can expose three evidence channels with intentionally different failure characteristics:

1. METADATA - fast and exact, but easily stripped
2. CONCEALED - unobtrusive and redundant, designed to survive common transforms
3. VISIBLE MINI-SAGE - optional, obvious, highly screenshot/format-conversion tolerant, but trivially removable or copyable

This provides useful diversity: screenshots commonly destroy metadata, recompression/resize may damage concealed transport, while a visible mini-code can survive both. Conversely, visible marks are easiest to intentionally remove. None should be treated as individually authoritative.

If multiple channels decode, disagreement should surface as diagnostic/conflict evidence rather than being silently resolved.

11. Mature operational model

If adopted broadly, SAGE should become boring infrastructure: stable encode/decode algorithms, media profiles, conformance vectors, and a tiny participant registry. Implementers integrate it once and largely forget about it.

```
CREATE / EDIT MEDIA
|
v
SAGE PRESERVE / UPDATE UNIQUE PARTICIPANT CHAIN
|
+--> optional visible mini-SAGE marker
|
v
MEDIA FILE
|
v
SAGE DECODE
|
v
SURVIVING PARTICIPANT EVIDENCE
|
v
OPTIONAL REGISTRY HANDSHAKE
```

The registry is not in the media-processing path and should ideally be mirrorable.

12. CODEX review points before normative changes

1. Inspect the latest repository and empirical transform-sweep work first.
2. Evaluate and, if accepted, replace AI_ID with PARTICIPANT_ID consistently across Core, profiles, .alg files, schemas, tests, examples, and white paper.
3. Evaluate PARTICIPANT_ID + optional opaque EXT_DATA_1..3 versus a generic extension array or PARTICIPANT_ID-only Core.
4. Implement/test the one-entry-per-participant invariant and remove-then-append update rule.
5. Define canonical equality and serialization under unique participant IDs.
6. Explicitly state chain order is relative last-touch order, not authenticated wall-clock chronology.

7. Do not add a mandatory timestamp unless implementation evidence reveals a requirement not met by ordering.
8. Update threat-model/trust wording so participant presence and final-recorded-participant are explicitly surviving evidence, not guaranteed complete history.
9. Reassess chain truncation semantics under unique-participant ordering.
10. Reassess cross-asset replay/provider verification implications if EVENT_ID is no longer mandatory.
11. Preserve that participant extension data are claims/opaque values, not SAGE-authenticated facts.
12. Keep the registry as participant discovery only; no central vendor provenance database.
13. Evaluate an optional visible mini-SAGE marker as a profile/encoder feature, including scan reliability at small sizes, corner placement, resize behavior, screenshot survival, and copy/replay risks.
14. Decide whether the visible marker belongs in Core, an image profile, or a separate visible-marker profile. Prefer profile-level treatment unless Core semantics truly require it.
15. Define how visible-marker disagreement with metadata/concealed evidence is surfaced. Do not make visible evidence authoritative.
16. Keep ECC/checksum/redundancy/placement in media profiles, not Core.
17. Add CODEX's own critique and reject/modify proposals that conflict with measured behavior or create unnecessary complexity.
18. If coherent, update relevant artifacts and commit with a clear commit message.

13. Design principle

SAGE is the handshake, not the conversation. Core should standardize only what independent implementations must agree on. Participant-specific meaning stays with the participant. The goal is lightweight surviving evidence, not a full forensic reconstruction engine.

End of revised handoff.

SAGE - CODEX HANDOFF v4 ADDENDUM

Reference implementation language roadmap

New discussion: SAGE should deliberately demonstrate that the Core/Profile logic translates cleanly across major programming ecosystems. Because the standard is intentionally small, maintaining a broad set of interoperable reference/example implementations should be realistic.

14. Organize by language, not framework

Reference implementations should be grouped primarily by programming language. Framework examples should consume those language libraries rather than becoming separate protocol implementations.

```
/reference
/c
/cpp
/csharp
/dart
/go
/java
/javascript
/kotlin
/php
/python
/rust
/swift
/typescript
```

Framework/application examples can then live separately:

```
/examples
/web-editor-php-js
/flutter-editor
/aspnet-core-example
/android-example
/ios-example
/node-example
```

Modern ASP should be treated as **ASP.NET Core / C#**, not classic .asp/VBScript. Flutter should be treated as a **Dart** implementation; Flutter is the application framework, not a separate SAGE language.

15. Proposed language roadmap

Tier	Language / Runtime	Primary value
1	JavaScript	Browser-side decode/encode, web editors, Node ecosystem
1	TypeScript	Typed modern JS applications and package ecosystem
1	PHP	CMS/server/web adoption; pairs naturally with browser JS
1	Python	Research, AI, testing, forensic tooling, CLI utilities
1	C	Portable native baseline; useful for bindings and embedded/native integration
1	C++	Image/video applications and native media pipelines
1	C# / .NET	Windows, desktop, services, ASP.NET Core
1	Java	JVM/server/cross-platform ecosystem
1	Dart	Flutter mobile/desktop reference editor
1	Rust	Modern systems/media tooling with strong safety properties
1	Swift	Native iOS/macOS integration

Tier	Language / Runtime	Primary value
2	Kotlin	Modern Android/JVM
2	Go	Servers, infrastructure, command-line tooling
2	Ruby	Rails and general web ecosystem
3	Objective-C	Legacy Apple integrations
3	VB.NET	Existing .NET software bases
3	Lua	Plugins/embedded application environments
3	Zig	Emerging native systems ecosystem

Practical first milestone: C, C++, C#, Dart, Java, JavaScript, PHP, Python, Rust, and Swift. Then add TypeScript, Kotlin, and Go. This gives SAGE broad coverage without requiring every ecosystem on day one.

16. Shared API and conformance behavior

Each implementation should expose approximately the same conceptual operations, even where language conventions differ:

```
sage_encode(...)
sage_decode(...)
sage_update_participant(...)
sage_read_metadata(...)
sage_read_concealed(...)
sage_write_metadata(...)
sage_write_concealed(...)
```

Every implementation must agree on canonical serialization, unique-participant update behavior, decoder states, profile recognition, damaged/conflict handling, and test-vector outputs.

The strongest demonstration is not that each library works by itself, but that independently written implementations can cross-decode each other's assets.

17. Cross-language conformance matrix

At minimum, CI should eventually exercise a rotating cross-language matrix rather than testing only encode/decode within the same implementation.

```
C encode -> Python decode
Python encode -> JavaScript decode
JavaScript encode -> Dart decode
Dart encode -> Java decode
Java encode -> C# decode
C# encode -> Rust decode
Rust encode -> Swift decode
Swift encode -> C++ decode
C++ encode -> PHP decode
PHP encode -> C decode
```

Shared malformed/damaged/conflicting assets should also be decoded by every implementation to prove that state semantics are portable, not merely the happy-path byte format.

18. CODEX actions added by this update

1. Propose a repository layout for multi-language reference implementations and framework examples.
2. Identify which existing Python/reference code can become the canonical behavioral oracle for test-vector generation without making Python normative.
3. Define a language-neutral conformance-vector format suitable for C, JS, PHP, Dart, JVM, .NET, Rust, Swift, and Go.
4. Recommend the first two additional ports after the current implementation based on implementation effort and test value.

5. Keep framework-specific integrations (ASP.NET Core, Flutter, Android, iOS, Node, web editor) outside Core semantics.
6. Update the white paper/implementation roadmap if this breadth of reference code is consistent with current project scope.
7. If coherent, commit the documentation/roadmap changes with the rest of the current SAGE update.

19. Adoption message

The repository itself should become proof that SAGE is language-neutral. A developer should be able to look at several small, interoperable implementations and conclude that SAGE is a protocol they can implement in their own stack without adopting a particular runtime, vendor, framework, or service.

SAGE - CODEX HANDOFF v5 ADDENDUM

Live registry API, nightly participant snapshots, caching, and mirrors

New direction: The participant registry may expose a tiny live HTTPS API for machine lookup while also publishing nightly complete participant dumps. The API must remain optional enrichment infrastructure, never a prerequisite for detecting SAGE evidence or an authority over media authenticity.

20. Optional live registry API

A decoder or application may resolve a PARTICIPANT_ID through a simple curl-friendly HTTPS endpoint. Local SAGE decoding always comes first; the registry only maps an opaque participant identifier to public registration information.

```
GET /api/v1/participants/{PARTICIPANT_ID}
```

Example response:

```
{
  "participant_id": "004821",
  "display_name": "Example Image Tool",
  "public_url": "https://vendor.example/sage",
  "status": "active"
}
```

A batch endpoint may be useful for resolving a recovered participant chain in one request:

```
GET /api/v1/participants?ids=004821,004822,004900
```

Normative trust boundary: a successful registry lookup means only that a PARTICIPANT_ID is registered and currently maps to a public registry record. It does not authenticate the media, extension values, chain completeness, or asset history.

21. Three registry operating modes

Applications should be able to operate in any of three modes without changing SAGE Core semantics:

```
LIVE
  decoder -> registry HTTPS API

CACHED
  decoder -> locally stored participant snapshot

OFFLINE
  decoder -> PARTICIPANT_ID only
```

Offline decoding must remain fully valid. Network access must never be silently enabled merely because registry resolution is available.

22. Nightly complete participant snapshots

The registry should publish a complete current participant dump on a nightly schedule. This is separate from ordinary infrastructure/database backups: the dump is a public interoperability artifact intended for caches, mirrors, reproducibility, and offline resolution.

Recommended current files:

```
participants.json
participants.csv
participants.sha256
manifest.json
```

Recommended dated archive:

```
/archive/  
2026-08-30-participants.json  
2026-08-30-participants.csv  
2026-08-30-participants.sha256  
2026-08-30-manifest.json
```

23. Snapshot manifest

Each nightly release should include a tiny machine-readable manifest so clients and mirrors can determine freshness and integrity without parsing the entire registry.

```
{  
  "registry_format_version": "1",  
  "generated_at": "2026-08-30T04:00:00Z",  
  "participant_count": 1842,  
  "participant_dump": "participants.json",  
  "sha256": "..."  
}
```

The checksum protects download integrity only; it must not be described as proof of media authenticity or participant truth.

24. Mirrorability and anti-centralization

Because SAGE is open and the registry is intentionally small, third parties should be able to mirror the registry using either the nightly dump or the public repository/database export format.

The architecture should avoid making a single SAGE-operated host a permanent protocol dependency. A decoder may use the canonical registry, a trusted mirror, a locally cached snapshot, or no registry at all.

If registry namespace governance later needs stronger guarantees, that should be solved independently of the media-evidence semantics rather than turning the lookup API into a trust oracle.

25. Suggested caching behavior

Clients using live lookup should cache participant records conservatively and should surface staleness where relevant. A nightly snapshot naturally provides a simple cache-generation boundary.

Possible client metadata:

```
registry_source  
snapshot_generated_at  
lookup_time  
cache_age  
registry_format_version
```

These are decoder/application diagnostics, not fields that need to be embedded into SAGE media.

26. Registry status semantics

CODEx should keep registry status intentionally small. Suggested initial values include:

Status	Meaning
active	Currently registered participant record
retired	Identifier remains resolvable but participant/tool is no longer active
revoked	Registry operator withdrew current standing; historical identifier remains resolvable

Status naming should be reviewed carefully so it does not imply certification. In particular, **revoked** should mean a registry-administration state, not that every asset carrying that participant ID is false or malicious.

27. CODEX actions added by this update

1. Evaluate a minimal curl-friendly read-only participant lookup API.
2. Keep live resolution optional and decoder network access disabled unless explicitly enabled by caller policy.
3. Define a nightly complete participant dump in JSON, CSV, checksum, and manifest form.
4. Define a dated snapshot archive and retention policy suitable for reproducibility without unnecessary operational burden.
5. Define a stable public dump schema that mirrors can consume independently of the live API implementation.
6. Consider efficient batch participant lookup for multi-entry chains.
7. Define minimal registry status semantics without implying certification/authentication.
8. Document live, cached, and offline registry modes.
9. Ensure registry/API failure never changes locally recovered SAGE PRESENT evidence into NOT_DETECTED or DAMAGED.
10. Keep registry backup/mirroring mechanics out of SAGE Core unless a truly protocol-level requirement emerges.
11. If coherent with current repository architecture, update specs/roadmap/tests and commit.

28. Registry design principle

Live when convenient. Cached when practical. Offline when necessary. The public registry is a directory for the handshake, while nightly complete dumps make that directory portable and resilient.

SAGE - CODEX HANDOFF v6 ADDENDUM

Anonymous public lookup with optional API keys for expanded rate limits

New refinement: Public participant lookup should remain usable without authentication. Optional API keys may be issued solely to provide higher request quotas, client-specific rate limits, and operational visibility for larger integrations. API keys must not unlock richer participant data or stronger trust semantics.

29. Anonymous-by-default lookup

The ordinary read-only participant registry endpoint should require no account, API key, OAuth flow, or developer registration.

```
curl https://registry.sage.example/api/v1/participants/004821
```

Anonymous clients receive the same public participant record as authenticated clients. Authentication changes capacity, not meaning or access to privileged provenance information.

30. Optional API keys for expanded rate limits

Applications needing sustained higher throughput may request an API key. The key exists for quota management and abuse control only.

```
Anonymous client
-> normal public rate limit

API-key client
-> expanded rate limit

Bulk/high-volume consumer
-> nightly participant snapshot / local cache
```

This creates a clean scaling path from hobbyist/desktop use to major platforms without making API registration a prerequisite for basic SAGE participation.

31. No privileged registry tier

An API key **MUST NOT** imply certification, participant verification, authenticated media provenance, or privileged access to hidden registry fields. The public registry dataset remains the public registry dataset.

The same PARTICIPANT_ID should resolve to the same public record whether the request is anonymous, API-key authenticated, served from a mirror, or resolved from the nightly dump.

32. Suggested rate-limit behavior

Exact quotas are operational policy and should not be hard-coded into SAGE Core. CODEX may define reference defaults for the canonical registry implementation.

Client mode	Example behavior
Anonymous	Generous baseline quota suitable for ordinary decoders and small applications
API key	Higher sustained quota and/or larger burst allowance
Nightly dump	Unlimited local lookups after snapshot download

A batch endpoint should have its own maximum IDs-per-request regardless of authentication to protect service stability.

33. Rate-limit response headers

The canonical API implementation may expose standard-style quota diagnostics so clients can back off cleanly:

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 742
X-RateLimit-Reset: 1788051600
Retry-After: 3600
```

Header naming may be aligned with prevailing HTTP conventions during implementation review; this is registry/API behavior, not media-format semantics.

34. Rate-limit exhaustion response

When quota is exceeded, the API should return HTTP 429 and point clients toward all three recovery paths: wait, use an API key for higher limits, or consume the current nightly registry snapshot.

```
{
  "error": "rate_limit_exceeded",
  "options": [
    "retry later",
    "request an API key for higher limits",
    "use the current registry snapshot"
  ]
}
```

35. API-key privacy and logging

Because keys are operational identifiers, the registry should minimize retention of request-level telemetry and document what is logged. Keys should not become identity-bearing media metadata and must never be embedded in SAGE records.

A key may identify an integration/account to the registry service for quota enforcement, but that relationship remains outside the SAGE media protocol.

36. API-key lifecycle

The registry implementation should support basic key lifecycle operations such as create, revoke, rotate, and optionally assign quota classes. These are ordinary service-administration features, not SAGE Core concepts.

Suggested minimal management model:

```
key_id
key_secret_hash
account_id
quota_class
status
created_at
last_used_at (optional)
```

Only a cryptographic hash of the secret should be stored where practical. CODEX should choose implementation details appropriate to the registry stack.

37. CODEX actions added by this update

1. Keep anonymous public participant resolution fully functional.
2. Add optional API-key authentication strictly as a higher-quota mechanism.
3. Ensure API keys do not expose richer participant data or stronger provenance assertions.
4. Define reference anonymous/keyed/batch rate-limit policies outside Core.
5. Return HTTP 429 with Retry-After and machine-readable guidance.
6. Encourage large-volume clients to use nightly dumps/local caches rather than continually querying the live API.

7. Define minimal key create/revoke/rotate administration in the registry implementation only.
8. Ensure API keys, secrets, account IDs, and service telemetry never enter SAGE media records.
9. Document privacy/log-retention expectations for authenticated registry clients.
10. Reconcile these API decisions with the nightly dump and mirror architecture from v5 and commit if coherent.

38. API scaling principle

Anonymous by default. Keys for throughput. Snapshots for scale. Authentication should manage server resources, not determine who is allowed to understand public SAGE participant records.